

Mycellius — Séance 9bis (1h)

*Correction CRUD - add DELETE and debug POST method to persist Database
Display Tags on GET request (API + FRONT) + Déconnexion*

Objectif

- Ajouter DELETE
- Correction persistance à la base de données lors d'un POST (Ajout d'un formulaire)
- modifier le front en conséquence

Les modifications vont impacter sur les fichiers suivants:

- WikiRepository
- InMemoryWiki
- DatabaseWikiRepository
- WikiService
- WikiPageController
- PageNotFoundException

Étape 1 — API - Autoriser DELETE dans la sécurité

1. Sécurité (RBAC)

Dans ta SecurityConfig, on a déjà GET/POST/PUT bridés par rôle, mais pas DELETE.

On va ajouter la règle DELETE comme POST/PUT (DEV/ADMIN) pour éviter 403 quand tu es DEV/ADMIN.

Ajoute la règle dans SecurityConfig.java:

```
.requestMatchers(HttpMethod.DELETE, "/api/v1/pages/**")  
.hasAnyRole("DEV", "ADMIN")
```

Étape 2 — API - Ajouter la suppression dans le “contrat” Repository

But : le service pourra supprimer sans savoir si c'est mémoire ou DB.

Dans le fichier : WikiRepository.java, rajoute à la fin de l'interface :

```
void deleteById(String id);
```

Étape 3 — API - Implémenter delete dans InMemoryWiki

But : si tu es en mode “mémoire”, DELETE marche aussi.

Dans le fichier : InMemoryWiki.java, ajoute la méthode :

```
@Override
public void deleteById(String id) {
    WikiPage removed = store.remove(id);
    if (removed == null) {
        throw new PageNotFoundException(id);
    }
}
```

Étape 4 — API - Implémenter delete dans le repository DB

But : si tu es en mode DB, suppression réelle en base.

Dans le fichier : DatabaseWikiRepository.java, ajoute :

```
@Override
public void deleteById(String id) {
    if (!pageJpa.existsById(id)) {
        throw new PageNotFoundException(id);
    }
    pageJpa.deleteById(id);
}
```

Étape 5 — API - Ajouter delete dans le service

Dans le fichier : WikiService.java, ajoute :

```
public void deletePage(String id) {
    repository.deleteById(id);
}
```

Étape 6 — API - Exposer l'endpoint DELETE

Dans le fichier : WikiPageController.java, ajoute :

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable String id) {
    wikiService.deletePage(id);
}
```

Étape 7 — API - Tester dans Postman

Refais un login après redémarrage, puis tu re-testes DELETE avec le nouveau token.

DELETE <http://localhost:8080/api/v1/pages/PAGE-00X>

Header : Authorization: Bearer <token admin/dev>

Attendu : 204.

FRONT WEB APP

A - Tags

Étape 1 — Mode edit : convertir res.tags en texte

Dans le fichier PageFormPage.jsx, remplacer :

```
setTagsText((res.tags ?? []).join(", "));
```

par:

```
setTagsText((res.tags ?? []).map(t => t.name).join(", "));
```

Étape 2 — Mode submit : envoyer des objets { value } (pas des strings)

Toujours dans le fichier PageFormPage.jsx, remplacer la construction des tags :

```
const tags = tagsText
  .split(",")
  .map((s) => s.trim())
  .filter(Boolean);
```

```
const payload = { ...form, tags };
```

par:

```
const tags = tagsText
  .split(",")
  .map(s => s.trim())
  .filter(Boolean)
  .map(name => ({ name }));
```

```
const payload = { ...form, tags };
```

Étape 3 — Afficher l'erreur renvoyée par l'API

Toujours dans le fichier PageFormPage.jsx, remplacer dans le catch{} :

```
setError("Erreur enregistrement");
```

par:

```
setError(e instanceof ApiError ? JSON.stringify(e.body) : "Erreur enregistrement");
```

Étape 4 — Après création : revenir à la liste avec notification

Toujours dans le fichier PageFormPage.jsx, remplacer dans le catch{} :

```
navigate("/pages");
```

par:

```
navigate("/pages", { state: { flash: { type: "success", message: `Page ${form.id} créée` } } });
```

B - Delete Page (côté API c'est bon)

On aimerait pouvoir supprimer les pages qu'on ne souhaite plus

Étape 1 — Ajout fonction onDelete

Dans le fichier PagesListPage.jsx, ajoute juste une fonction onDelete (avant le return(...)), puis un bouton "Supprimer" dans le . :

```
async function onDelete(id) {
  if (!window.confirm(Supprimer `${id} ?`)) return;

  try {
    await apiRequest(`/api/v1/pages/${id}`, { method: "DELETE", token });
    setPages(prev => prev.filter(p => p.id !== id)); // update UI
  } catch (e) {
    if (e instanceof ApiError && e.status === 401) {
      logout();
      navigate("/login");
      return;
    }
    if (e instanceof ApiError && e.status === 403) {
      navigate("/forbidden");
      return;
    }
    setError(e instanceof ApiError ? JSON.stringify(e.body) : "Erreur suppression");
  }
}
```

Étape 2 — Ajout bouton "Supprimer" suivant le rôle du User

Dans le map, juste après le <Link ...>, ajoute le bouton (DEV/ADMIN) :

```
{(role === "DEV" || role === "ADMIN") && (
  <button style={{ marginLeft: 10 }} onClick={() => onDelete(p.id)}>
    Supprimer
  </button>
)}
```

C - Déconnexion

On aimerait pouvoir supprimer se déconnecter

Étape 1 — Ajout bouton déconnexion

Dans PagesListPage.jsx, récupère une fonction logout qui fait vraiment la déconnexion.

Ton useAuth() expose déjà logout

Ajoute un bouton dans le header (dans le <div style={{ marginBottom: 12 }}>, par ex juste sous “Rôle” :

```
<button
  style={{ marginLeft: 12 }}
  onClick={async () => {
    await logout();
    navigate("/login");
  }}
>
  Déconnexion
</button>
```

FRONT MOBILE APP

A - Delete

Étape 1 — suppression depuis `PagesListScreen.js`

Dans le fichier `PagesListPage.jsx`, ajoute la fonction `delete` :

```
async function onDelete(id) {
  try {
    await apiRequest(`/api/v1/pages/${id}`, { method: "DELETE", token });
    setPages(prev => prev.filter(p => String(p.id) !== String(id)));
  } catch (e) {
    if (e instanceof ApiError && e.status === 401) { await logout(); return; }
  }
}
```

Étape 2 — Ajout du bouton "Supprimer"

Dans le `return`, ajoute un bouton (DEV/ADMIN) :

```
renderItem=(({ item }) => (
  <View style={{ paddingVertical: 10 }}>
    <TouchableOpacity onPress={() => navigation.navigate("PageDetail", { id: item.id })}>
      <Text>{item.title} {item.id}</Text>
    </TouchableOpacity>

    {(role === "DEV" || role === "ADMIN") && (
      <Button title="Supprimer" onPress={() => onDelete(item.id)} />
    )}
  </View>
)}
```

B - Déconnexion

Étape 1 — Ajout d'un bouton de déconnexion (`AppNavigator`)

Avec React Navigation (native stack), tu mets le bouton "Déconnexion" dans le header via `headerRight`. Et pour "Détail", le retour vers la liste se fait via la flèche back du header. Comme `AppNavigator` a déjà accès à `useAuth()`, c'est l'endroit idéal.

Dans `mobile/App.js` :

Remplace ta partie `token ? ... : ...` par une version avec `screenOptions + headerRight` :

Dans le fichier PagesListScreen.jsx, tu as déjà "logout" par ex sous "Rafraîchir" :

```
<Button title="Déconnexion" onPress={logout} />
```

et :

```
import { Button } from "react-native";
```

Comme ta navigation dépend de token dans App.js, dès que logout() met token à null, tu reviens automatiquement sur l'écran Login.

Oui il y a plusieurs endroits pour mettre un menu sur React Native, en haut, en bas, et chaque composant est une stack.

Je vous laisse vous amuser à choisir ce que vous préférez et supprimer ce que vous ne gardez pas.

On remarque un autre bug, les tags ne s'affichent pas... OMG !!!!!

Objectif 2

- Afficher les Tags côté API et côté FRONT web et mobile

Les modifications vont impacter sur les fichiers suivants:

- WikiPageDtoMapper

Étape 1 — API - Affichage des tags "cachés" dans la Page "Liste"

C'est l'API qui cache volontairement les tags dans la réponse.

Dans ton fichier WikiPageDtoMapper, tu as cette ligne :

```
@Mapping(target = "tags", expression = "java(java.util.List.of())")
```

Ça veut dire :

“peu importe les tags réels, renvoyer toujours une liste vide”. Donc tu auras toujours tags: [].

On va corriger dans ton fichier WikiPageDtoMapper en supprimant ou commentant les 2 lignes :

```
@Mapping(target = "createdAt", ignore = true)
@Mapping(target = "tags", expression = "java(java.util.List.of())")
```

Et ajoute juste à l'interface une petite méthode dans ce mapper pour expliquer à MapStruct comment convertir un Tag en String :

```
default String map(Tag tag) {
    return tag == null ? null : tag.getValue();
}
```

et laisse toResponse “simple” :

```
WikiPageResponse toResponse(WikiPage page);
```

Etape 2 — Check

Redémarre l'API et refais un GET :

```
GET /api/v1/pages/PAGE-007
```

=> Tu dois voir les tags remonter.

Etape 3 — API - Persistence en mode "edit"

Contrôler : le PUT doit utiliser le même DTO que le POST (celui qui a tags)

Notre contrôleur n'a pas de PUT.

Donc en "edit" côté web, tu appelles PUT /api/v1/pages/{id}... mais l'API ne l'implémente pas, donc rien ne peut persister (tags compris).

1. Ajouter updatePage dans WikiService

- On va vérifier que la page existe, sinon PageNotFoundException
- On va sauvegarder via repository.save(...) (c'est là que les tags sont persistés)

Dans WikiService, ajoute ceci (par exemple entre createPage et getPageById) pour garantir que les tags reçus sont bien passés à repository.save(), qui les persiste.) :

```
public WikiPage updatePage(String id, WikiPage page) {
    if (page == null) {
        throw new IllegalArgumentException("La page est obligatoire");
    }

    WikiPage existing = repository.getByld(id);
    if (existing == null) {
        throw new PageNotFoundException(id);
    }

    // l'id de l'URL est la source de vérité
    page.setld(id);

    // on conserve createdAt si le mapper l'a ignoré (ce qui est ton cas)
    if (page.getCreatedAt() == null) {
        page.setCreatedAt(existing.getCreatedAt());
    }

    return repository.save(page); // persiste aussi les tags
}
```

2. Ajouter le PUT dans WikiPageController

Dans ton WikiPageController, ajoute :

```
@PutMapping("/{id}")
public WikiPageResponse update(
    @PathVariable String id,
    @Valid @RequestBody CreateWikiPageRequest request
) {
    WikiPage updated = wikiService.updatePage(id, mapper.toDomain(request));
    return mapper.toResponse(updated);
}
```

3. Tester en Postman

PUT http://localhost:8080/api/v1/pages/PAGE-007

Body :

```
{
  "id": "PAGE-007",
  "title": "Titre",
  "content": "Contenu",
  "tags": [ { "name": "tag7" }, { "name": "nouveau" } ]
}
```

=> **Attendu** : 200, puis GET doit montrer "tags":["tag7","nouveau"].

Etape 4 — Affichage dans le front web et mobile

FRONT WEB

1. Afficher les tags dans la liste (PagesListPage.jsx)

Dans ton return, dans le après le <link> (ou là où tu affiches chaque page), ajoute :

```
{(p.tags?.length ?? 0) > 0 && (
  <div style={{ fontSize: 12, opacity: 0.8 }}>
    Tags : {p.tags.join(", ")}
  </div>
)}
```

2. Afficher les tags dans le détail (PageDetailPage.jsx)

Sous le titre par exemple, ajoute :

```
{(page.tags?.length ?? 0) > 0 && (
  <p>Tags : {page.tags.join(", ")}</p>
)}
```

3. Edit mode : pré-remplir le champ tags + envoyer le bon JSON (PageFormPage.jsx)

Dans ton load() du useEffect "mode edit", tu as cette ligne :

```
setTagsText((res.tags ?? []).map(t => t.name).join(", "));
```

remplace là par :

```
setTagsText((res.tags ?? []).join(", "));
```

FRONT MOBILE

Modifs minimales côté mobile (Expo / React Native) pour POST + DELETE + PUT + tags (API attend tags: [{ "name": "..."}]).

1. Nouveau screen : création d'une page

Crée le fichier : mobile/src/screens/NewPageScreen.js

```
import React, { useState } from "react";
import { View, Text, TextInput, Button } from "react-native";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";

export default function NewPageScreen({ navigation }) {
  const { token, role, logout } = useAuth();

  const [id, setId] = useState("PAGE-001");
  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [tagsText, setTagsText] = useState("tag1,tag2");
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  async function onCreate() {
    setError(null);
    setLoading(true);

    const tags = tagsText
      .split(",")
      .map(s => s.trim())
      .filter(Boolean)
      .map(name => ({ name })); // IMPORTANT: API attend TagRequest.name

    try {
      await apiRequest("/api/v1/pages", {
        method: "POST",
        token,
        body: { id, title, content, tags }
      });
      navigation.goBack(); // retour liste
    } catch (e) {
      if (e instanceof ApiError && e.status === 401) { await logout(); return; }
      setError(e instanceof ApiError ? JSON.stringify(e.body) : "Erreur création");
    } finally {
      setLoading(false);
    }
  }

  if (!(role === "DEV" || role === "ADMIN")) {
    return (
      <View style={{ padding: 16 }}>
        <Text>Accès interdit (DEV/ADMIN)</Text>
      </View>
    );
  }
}
```

```

    }

    return (
      <View style={{ padding: 16, gap: 10 }}>
        <Text>ID (format PAGE-001)</Text>
        <TextInput value={id} onChangeText={setId} style={{ borderWidth: 1, padding: 8 }} />

        <Text>Titre</Text>
        <TextInput value={title} onChangeText={setTitle} style={{ borderWidth: 1, padding:
8 }} />

        <Text>Contenu</Text>
        <TextInput
          value={content}
          onChangeText={setContent}
          multiline
          style={{ borderWidth: 1, padding: 8, minHeight: 120 }}
        />

        <Text>Tags (virgules)</Text>
        <TextInput value={tagsText} onChangeText={setTagsText} style={{ borderWidth: 1,
padding: 8 }} />

        <Button title={loading ? "Création..." : "Créer"} onPress={onCreate} disabled={loading}
      />
      {error && <Text style={{ marginTop: 8 }}>{error}</Text>}
    </View>
  );
}

```

2. Navigation : ajouter l'écran "NewPage"

Dans mobile/App.js: importe et ajoute la route.

En haut :

```
import NewPageScreen from "../src/screens/NewPageScreen";
```

Dans la partie "token présent"v entre "Pages" et "PageDetail" :

```
<Stack.Screen name="NewPage" component={NewPageScreen} options={{ title: "Créer une page" }} />
```

3. Liste : bouton "Créer" (DEV/ADMIN) + refresh au retour

Dans mobile/src/screens/PagesListScreen.js :

Ajoute un bouton :

```

{(role === "DEV" || role === "ADMIN") && (
  <Button title="Créer une page" onPress={() => navigation.navigate("NewPage")} />
)}

```

Et pour recharger quand on revient de "NewPage", remplace ton useEffect par :

```
useEffect(() => {  
  const unsub = navigation.addListener("focus", loadList);  
  return unsub;  
}, [navigation, token]);
```

4. Détail : bouton "Supprimer" (DEV/ADMIN)

Dans mobile/src/screens/PageDetailScreen.js :

Ajoute role :

```
const { token, role, logout } = useAuth();
```

Ajoute la fonction :

```
async function onDelete() {  
  try {  
    await apiRequest(`/api/v1/pages/${id}`, { method: "DELETE", token });  
    navigation.goBack();  
  } catch (e) {  
    if (e instanceof ApiError && e.status === 401) { await logout(); return; }  
  }  
}
```

Ajoute le bouton dans le rendu (en bas) :

```
{(role === "DEV" || role === "ADMIN") && (  
  <Button title="Supprimer" onPress={onDelete} />  
)}
```

5. Affichage tags

Dans PageDetailScreen, si la réponse API renvoie tags: ["tag1", "tag2"] :

```
{(page.tags?.length ?? 0) > 0 && <Text>Tags : {page.tags.join(", ")}</Text>}
```

=> C'est tout pour POST + DELETE + tags.

6. Ajout "EDIT (PUT)" côté mobile

On va créer l'écran d'édition

Fichier : mobile/src/screens/EditPageScreen.js

```
import React, { useEffect, useState } from "react";  
import { View, Text, TextInput, Button } from "react-native";  
import { apiRequest, ApiError } from "../../api/apiClient";  
import { useAuth } from "../../auth/AuthContext";
```

```

export default function EditPageScreen({ route, navigation }) {
  const { id } = route.params;
  const { token, role, logout } = useAuth();

  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [tagsText, setTagsText] = useState("");
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    (async () => {
      try {
        const page = await apiRequest(`/api/v1/pages/${id}`, { token });
        setTitle(page.title ?? "");
        setContent(page.content ?? "");
        setTagsText((page.tags ?? []).join(", ")); // API renvoie List<String>
      } catch (e) {
        if (e instanceof ApiError && e.status === 401) { await logout(); return; }
        setError("Erreur chargement");
      }
    })();
  }, [id, token]);

  async function onSave() {
    setError(null);
    setLoading(true);

    const tags = tagsText
      .split(",")
      .map(s => s.trim())
      .filter(Boolean)
      .map(name => ({ name })); // API attend TagRequest.name

    try {
      await apiRequest(`/api/v1/pages/${id}`, {
        method: "PUT",
        token,
        body: { id, title, content, tags }
      });
      navigation.goBack(); // retour détail
    } catch (e) {
      if (e instanceof ApiError && e.status === 401) { await logout(); return; }
      setError(e instanceof ApiError ? JSON.stringify(e.body) : "Erreur update");
    } finally {
      setLoading(false);
    }
  }

  if (!(role === "DEV" || role === "ADMIN")) {

```

```

    return <View style={{ padding: 16 }}><Text>Accès interdit (DEV/ADMIN)</Text></View>;
  }

  return (
    <View style={{ padding: 16, gap: 10 }}>
      <Text>ID : {id}</Text>

      <Text>Titre</Text>
      <TextInput value={title} onChangeText={setTitle} style={{ borderWidth: 1, padding: 8 }} />

      <Text>Contenu</Text>
      <TextInput
        value={content}
        onChangeText={setContent}
        multiline
        style={{ borderWidth: 1, padding: 8, minHeight: 120 }}
      />

      <Text>Tags (virgules)</Text>
      <TextInput value={tagsText} onChangeText={setTagsText} style={{ borderWidth: 1, padding:
8 }} />

      <Button title={loading ? "Enregistrement..." : "Enregistrer"} onPress={onSave}
disabled={loading} />
      {error && <Text>{error}</Text>}
    </View>
  );
}

```

7. Ajouter la route dans App.js

Dans mobile/App.js :

Ajoute l'import :

```
import EditPageScreen from "../src/screens/EditPageScreen";
```

dans le Stack (token présent) :

```
<Stack.Screen name="EditPage" component={EditPageScreen} options={{ title: "Éditer" }} />
```

8. Ajouter un bouton “Éditer” dans le détail

Dans mobile/src/screens/PageDetailScreen.js, récupère role si pas déjà, puis ajoute un bouton :

```

    {(role === "DEV" || role === "ADMIN") && (
      <Button title="Éditer" onPress={() => navigation.navigate("EditPage", { id })} />
    )}

```


9. Recharger le détail après retour d'édition (focus)

Dans PageDetailScreen.js, au lieu d'un useEffect simple, fais un "focus reload" :

Remplace :

```
useEffect(() => {  
  (async () => {  
    try {  
      const res = await apiRequest(`/api/v1/pages/${id}`, { token });  
      setPage(res);  
    } catch (e) {  
      if (e instanceof ApiError && e.status === 401) { await logout(); return; }  
    }  
  })();  
}, [id, token]);
```

par :

```
useEffect(() => {  
  const unsub = navigation.addListener("focus", async () => {  
    try {  
      const res = await apiRequest(`/api/v1/pages/${id}`, { token });  
      setPage(res);  
    } catch (e) {  
      if (e instanceof ApiError && e.status === 401) { await logout(); }  
    }  
  });  
  
  return unsub;  
}, [navigation, id, token]);
```

Ici, navigation.addListener("focus", ...) recharge la page :

A chaque fois que l'écran redevient actif (focus), donc :

- première arrivée
- retour depuis "EditPage"
- retour depuis n'importe quel autre écran

=> En gros, il y a recharge à chaque retour sur l'écran

A l'issu de ce TD, vous avez un front Web et mobile fonctionnels !

A vous de jouer pour le style !